

Conocimientos en BBDD

Optimización de Base de Datos (SQL / PL/SQL) Tienes una tabla orders con 10 millones de registros y una tabla customers con 500,000 registros. Necesitas ejecutar frecuentemente la siguiente consulta para encontrar los pedidos de clientes de 'México', pero es muy lenta.
SQL

```
SELECT o.order_id, o.order_date, c.customer_name, o.total_amount
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
WHERE c.country = 'México';
```

Describe al menos tres estrategias diferentes que implementarías para optimizar el rendimiento de esta consulta.

Primero es saber que tipo de servidor de SQL / PL SQL seria, por qué así queda imposible de predecir que se puede hacer, la información es muy poca, saber si el servidor se puede tocar para optimizar en el proceso, por qué veo que el principal problema es el de optimización y el segundo sería de organización de la sentencia y además de esto que sea un procedimiento almacenado:

```
CREATE PROCEDURE sp_get_orders_by_country
    @country VARCHAR(100)
AS
BEGIN
    SELECT
        o.order_id,
        o.order_date,
        c.customer_name,
        o.total_amount
    FROM orders o
    LEFT OUTER JOIN customers c on o.customer_id = c.customer_id
    WHERE lower(c.country) like lower(@country);
END;
```

Para obtener solo los que **Customer** esten en **México**, Left outer join esto reduce en un indice exponencial la consulta: <https://www.datacamp.com/tutorial/left-join-vs-left-outer-join>

Entonces primero realizar un procedimiento almacenado, segundo crear un left outer join, tercero las estrategias de segmentación que se podrían usar de optimización.

Cost-Based Optimizer (CBO) del motor está ciego, como Desarrollador - DBA, la primera acción antes de alterar estructuras debería ser asegurar que las estadísticas de las tablas estén actualizadas para que el motor decida correctamente si usar un Hash Join o un Nested Loop Join como, ya va:

Primera estrategia

La manera más sencilla de explicar cómo optimizar esta base de datos puede ser por tres procedentes, primero mediante DDL o sea mediante la creación de tablas parametrizadas, índices estratégicos, vistas materializadas y lo más importante particionamiento de tablas se puede desarrollar en frío, que quiere decir esto, ya que la tabla sea muy grande lo cual particionarse por partes, primero midiendo el parámetro de ejecución ((Job process) x (Time Execution)) e / Numero de registros (totales) = S(value question).

Cost-Based Optimizer (CBO)

Por ejemplo si tenemos una tabla, con 20 millones de registros que se demora 15 segundos hacer y proceso almacenado es sencillo, El cálculo los clientes maneras:

(10.000.000 registro * 15)

$$\frac{\text{-----}}{500.000 \text{ registro}} = \frac{(150.000.000)e}{500.000} = \frac{407742274268856785304043120}{500.000 \text{ Record}} =$$

Sabiendo esto podríamos decir que cada proceso de base de datos utiliza 845,541.142 ciclos de reloj por segundo que es una carga altísima en ese procesamiento de un dato.

Entonces sería primero:

En la tabla customers: Crear un Covering Index (índice de cobertura), si indexamos el país, pero incluimos las otras columnas requeridas en el SELECT y el JOIN, el motor leerá directamente del índice sin tener que ir a los bloques de datos de la tabla.

```
CREATE INDEX idx_customers_country_cov  
ON customers (country, customer_id, customer_name);
```

En la tabla orders: sería vital asegurarse de que la llave foránea esté indexada., sin este índice, por cada cliente de México, el motor escanearía los 20 millones de registros de órdenes.

```
CREATE INDEX idx_orders_customer_id ON orders (customer_id);
```

Además podríamos evaluar en la consulta sea Upper o ToLower para agilizar la petición:

Esto en SQL

```
SELECT o.order_id, o.order_date, c.customer_name, o.total_amount
```

```
FROM orders o
Left outer join customers c ON o.customer_id = c.customer_id
WHERE unaccent(lower(c.country)) like unaccent(lower('%México%')) **;
```

**** Unaccent: Únicamente aplicable en la base de datos desde el 2012, 2017 en SQL server, pero en Postgress y oracle aplicable desde versiones de 2010 o Oracle 12i como SUBSTRING collate binary_la (siendo la = latin).**

Por que usamos unaccent por qué nosotros evitamos los signos especiales, en este caso é o É son simbolos especiales, aunque deberia estar en la base de datos realmente indexada, participionada NO deberia aparecer México sino el código del país.

Segunda estrategia:

Crear una Materialized View (Vista Materializada) pero si esto es usado en Oracle, o Postgress, al desconocer que motor y que version es complejo decidir, a diferencia de una vista normal que ejecuta el query en tiempo real, la vista materializada guarda el resultado físico en disco.

```
CREATE MATERIALIZED VIEW mv_mexico_orders
BUILD IMMEDIATE
REFRESH FAST ON COMMIT -- O REFRESH COMPLETE según el motor y la necesidad
AS
SELECT o.order_id, o.order_date, c.customer_name, o.total_amount
FROM orders o
Left join customers c ON o.customer_id = c.customer_id
WHERE c.country = 'México';
```

O crear un procedimiento almacenado si es muy usado, obviamente sin que sea una transaccion si se hacce demasiado veces en el motor de BD:

```
create procedure sp_get_orders_by_country
    @country nvarchar(100)
as
begin
    set nocount on;

    select o.order_id, o.order_date, c.customer_name, o.total_amount
    from orders o
    Left outer join customers c
        on o.customer_id = c.customer_id
    where c.country = @country;
end
```

Y la ejecución de este SP:

```
exec sp_get_orders_by_country 'México';
```

Aunque si buscas en chatGPT vas a ver otra alternativa

```
create procedure sp_get_orders_by_country
    @country nvarchar(100),
    @page int = 1,
    @pagesize int = 50
as
begin
    set nocount on;
    select o.order_id, o.order_date, c.customer_name, o.total_amount
    from orders o
    Left outer join join customers c
        on o.customer_id = c.customer_id
    where c.country = @country
    order by o.order_date desc
    offset (@page-1)*@pagesize rows
    fetch next @pagesize rows only;
end
```

El problema de la paginación, es que deja al servicio de la base de datos, pero llega a saturar la aplicación debido a que no lo hace de manera lazy loader sino de tipo transacción.

Tercera estrategia (la que menor me gusta):

Implementar particionamiento por Lista (List Partitioning) en la tabla **customers** basado en la columna country, de esta forma, todos los clientes de México estarán en un segmento físico aislado.

A nivel avanzado (en motores como Oracle o PostgreSQL), puedes aplicar Reference Partitioning (Particionamiento de Referencia), que quiere decir esto, si es una consulta demasiado masiva podría restringir el proceso a cada país, esto se hace que la tabla orders herede automáticamente la partición de la tabla **customers** a través de la llave foránea, cuando el motor busque las órdenes de México, Estoy seguro que esto nadie lo leerá... aplicará Partition Pruning (Poda de particiones), ignorando físicamente los millones de registros de otros países, reduciendo drásticamente el I/O en disco.

Pero siendo que tenemos un índice de 500.000 registros, sería mejor optimizar la base de datos, retirar el cache del servidor (Buffer Pool o alternativas similares), asegúrese de que el

parámetro DB_CACHE_SIZE esté adecuadamente configurado para albergar una parte significativa de los 500k registros más utilizados.